

Highlights

A Formalized Foundation for Research in Untyped Lambda Calculus

Chris Anto Fröschl, Prof. Matthias Güzdemann

- First machine-checked formalization of Crole's five equivalent definitions of alpha equivalence.
- An isomorphism, formalized in Lean 4, between a de Bruijn representation and the quotient of named terms by alpha equivalence, following Norrish's proof pearl.
- A roadmap toward a formalized foundation for untyped lambda calculus research, including further binding representations and a "formalized Barendregt".

A Formalized Foundation for Research in Untyped Lambda Calculus

Chris Anto Fröschl^{a,*}, Prof. Matthias Güzemann^{a,*}

^aMunich University of Applied Sciences, Department of Computer Science and Mathematics, Lothstraße 34, Munich, D-80335, Bavaria, Germany

ARTICLE INFO

Keywords:

α -equivalence
Atom
 λ -expression
Permutation action
Renaming
Variable binding
de Bruijn indices
formal verification
proof assistant
Lean
2020 MSC: 03B70
68V15
68V20

ABSTRACT

TODO Intro CSLib, Crole 5 equivs, dB = QAEQUIV and what it builds on Derived results from it, "Formalized Barendregt" Perhaps even benchmark like lambda n ways?

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum. Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

1. Introduction

Variable binding is the recurring technical obstacle in any formalization touching the lambda calculus: essentially every representation choice (named terms modulo alpha equivalence, de Bruijn indices, locally nameless terms, higher-order abstract syntax, nominal sets, ...) trades one set of proof obligations for another, and results proved for one representation rarely transfer automatically to the others. This has led to a large but fragmented body of formalization work, often redone from scratch for each new proof assistant or each new project, with little of it packaged as a reusable library.

CSLib is an effort to build such a reusable, foundational library for computer science results in the Lean 4 theorem prover, in the spirit of Mathlib for mathematics [3]. This paper reports on two contributions to CSLib's treatment of the untyped lambda calculus:

1. a machine-checked formalization of Crole's result that five common literature definitions of alpha equivalence on named lambda terms are equivalent [4], which to our knowledge had not previously been formalized in a proof assistant; and
2. an isomorphism, following Norrish's proof pearl [5], between the quotient of named terms by a chosen alpha equivalence and a de Bruijn representation, connecting the named and nameless ends of CSLib's binding infrastructure.

We present both results as part of a longer-term plan: to grow CSLib into an idiomatic, well-reviewed, and maintained foundation that other researchers can build on directly, rather than reformalizing lambda calculus basics for each new project. Section 6 sketches this plan in more detail, including further equivalences in the style of Crole's for SK combinator calculus and other binding representations, and a longer-term goal of formalizing a substantial part of Barendregt's reference treatment of the untyped lambda calculus [2].

Outline. Section 2 introduces CSLib and its current named representation of the untyped lambda calculus. Section 3 presents the formalization of Crole's alpha-equivalence equalities. Section 4 presents the isomorphism with the de Bruijn representation. Section 5 discusses related formalization efforts, Section 6 lays out planned extensions, and Section 7 concludes.

*Corresponding author

 chris.froeschl@hm.edu (C.A. Fröschl); matthias.guedemann@hm.edu (Prof.M. Güzemann)
ORCID(s): 0009-0006-3562-5395 (C.A. Fröschl); 0000-0002-1002-6023 (Prof.M. Güzemann)

2. CSLib and its named representation of the lambda calculus

2.1. CSLib

CSLib is a community-maintained library of foundational computer science formalizations for the Lean 4 theorem prover, built on top of Mathlib. [3] Its goal is to provide idiomatic, well-reviewed, and actively maintained formalizations of standard results, so that later research projects can build directly on a shared, trusted base rather than re-deriving basic definitions and lemmas from scratch. The work reported in this paper concerns CSLib’s treatment of the untyped lambda calculus, and in particular its representation(s) of variable binding.

2.2. Named terms

CSLib currently represents untyped lambda terms in a standard named style: variables are drawn from a countably infinite set of atoms, and terms are built from variables, application, and abstraction. TODO dummy math

Definition 2.1 (Named lambda terms). Let Var be a countably infinite set of variables. The set Lam of named lambda terms is generated by

$$t ::= x \mid tt \mid \lambda x. t \quad (x \in \text{Var}).$$

On top of this definition, CSLib provides the usual auxiliary notions needed to state and prove properties of the calculus: free variables, capture-avoiding substitution, and a baseline notion of alpha equivalence used as the reference point against which Crole’s alternative definitions are shown equivalent in Section 3.

3. Formalizing Crole’s alpha-equivalence equalities

Crole [4] gives five definitions of alpha equivalence on named lambda terms, arising from different intuitions about what it means to rename bound variables (via explicit substitution, via swapping, via simultaneous renaming of bound variable occurrences, ...), and proves that all five coincide. The proof is unremarkable to state on paper, but formalizing it exposes a number of routine-but-fiddly inductive arguments about substitution and renaming under binders that are easy to state informally and easy to get subtly wrong when made fully precise — which is precisely the kind of result CSLib aims to package once and for all.

3.1. The five definitions

3.2. Equivalence of the five definitions

TODO theorems + proof links + sketch

Theorem 3.1 (Crole [4], formalized). *The five relations of Section 3.1 coincide as relations on Lam .*

3.3. Formalization notes

TODO extra work due to the paper handing to the literature

4. An isomorphism with de Bruijn terms

TODO: fill in the actual de Bruijn development being reused (source/authorship, what was already in CSLib/Mathlib vs. what was added), the precise choice of which of the five equivalent alpha equivalences is quotiented by, and the isomorphism statement and proof sketch, following Norrish’s proof pearl.

De Bruijn indices remove the need for alpha equivalence altogether by replacing bound variable names with numerals counting binder depth. Norrish’s proof pearl [5] demonstrates that, despite a reputation for being awkward to reason about, a de Bruijn representation supports clean, structurally recursive proofs of the standard metatheory of the lambda calculus once the representation is set up carefully. We reuse an existing de Bruijn development and show that it is isomorphic to the quotient of named terms Lam by (any one of) the alpha equivalences of Section 3.

4.1. De Bruijn terms

Definition 4.1 (De Bruijn terms). The set Lam_{db} of de Bruijn lambda terms is generated by

$$t ::= n \mid tt \mid \lambda. t \quad (n \in \mathbb{N}).$$

4.2. The isomorphism

Theorem 4.2. $\text{Lam}/=_{\alpha} \cong \text{Lam}_{\text{db}}$.

5. Related work

Formalizing variable binding has a long history across proof assistants, and several general-purpose approaches exist alongside the direct, elementary style used in CSLib. Nominal techniques [6], as implemented in Nominal Isabelle [7], provide binder-generic reasoning principles via permutations and freshness, at the cost of adopting a dedicated meta-theory. Locally nameless representations [1] aim to make binder-heavy formalizations more tractable or more automatically generated from a specification, respectively. Higher-order abstract syntax sidesteps binder representation issues by delegating them to the proof assistant's own binding mechanism, again at the cost of restricting which proof techniques remain available (e.g. induction principles).

CSLib's current approach is deliberately more elementary: named terms and de Bruijn terms are both represented directly as first-order inductive types, with the relationships between representations (such as the isomorphism of Section 4) proved explicitly rather than assumed via a shared meta-level binding mechanism. This trades some proof automation for representations that are simple to state, and results that transfer cleanly between representations by construction, which fits CSLib's goal of a reusable foundation rather than a self-contained framework.

6. Future work

TODO sketch general net of equivalences in Proof pearls aswell (image)

The two results of this paper are intended as the first pieces of a larger, ongoing effort within CSLib to build a formalized foundation for research on untyped lambda calculus. We outline the planned directions below.

6.1. Further equivalence results

Crole-style results, showing several natural but superficially different definitions of the same notion to coincide, are not unique to alpha equivalence. We plan analogous formalizations for SK combinator calculus and other binding representations (locally nameless terms, nominal terms), each intended to slot into CSLib alongside the results of this paper and to be connected to them by explicit isomorphisms in the style of Section 4.

6.2. A formalized Barendregt

Barendregt's *The Lambda Calculus: Its Syntax and Semantics* [2] remains the standard reference for the untyped lambda calculus, and a substantial fraction of its content — confluence, standardization, the theory of reduction, combinatory completeness, and the associated notions of models — has never been formalized in a single, coherent development. We see CSLib's binding infrastructure as a natural basis for such an effort, which we tentatively refer to as a "formalized Barendregt". Beyond its intrinsic value as a reusable library, a sufficiently complete formalized Barendregt would put open or only recently resolved conjectures discussed in Barendregt's later survey work within reach of machine-checked treatment.

6.3. Maintenance and reuse

Throughout, the intent is for this work to be maintained as part of CSLib rather than as a standalone, one-off development: subject to code review, kept up to date with CSLib and Mathlib, and structured so that later projects can depend on it directly.

7. Conclusion

We have presented CSLib's current named representation of the untyped lambda calculus, a formalization of Crole's result that five notions of alpha equivalence coincide, and an isomorphism, following Norrish, between the quotient of named terms under alpha equivalence and a de Bruijn representation. Together these results connect two of the standard approaches to variable binding within a single, reusable Lean 4 library, and form the starting point for a longer-term effort to build a maintained, idiomatic foundation for research on untyped lambda calculus, including further binding representations and a formalized treatment of Barendregt's reference text.

Data availability

The Lean 4 formalization discussed in this paper is developed as part of CSLib and is publicly available at <https://github.com/leanprover/cslib>.

TODO consider with zenodo

References

- [1] Aydemir, B., Charguéraud, A., Pierce, B.C., Pollack, R., Weirich, S., 2008. Engineering formal metatheory, in: Proceedings of the 35th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, Association for Computing Machinery, New York, NY, USA. p. 3–15. URL: <https://doi.org/10.1145/1328438.1328443>, doi:10.1145/1328438.1328443.
- [2] Barendregt, H.P., 1985. The lambda calculus - its syntax and semantics. volume 103 of *Studies in logic and the foundations of mathematics*. North-Holland.
- [3] Barrett, C., Chaudhuri, S., Montesi, F., Grundy, J., Kohli, P., de Moura, L., Rademaker, A., Yingchareonthawornchai, S., 2026. Cslib: The lean computer science library. URL: <https://arxiv.org/abs/2602.04846>, arXiv:2602.04846.
- [4] Crole, R.L., 2012. Alpha equivalence equalities. *Theoretical Computer Science* 433, 1–19. URL: <https://www.sciencedirect.com/science/article/pii/S0304397512000667>, doi:10.1016/j.tcs.2012.01.030.
- [5] Norrish, M., Vestergaard, R., 2007. Proof pearl: De bruijn terms really do work, in: Schneider, K., Brandt, J. (Eds.), *Theorem Proving in Higher Order Logics*, Springer Berlin Heidelberg, Berlin, Heidelberg. pp. 207–222.
- [6] Pitts, A.M., 2013. Nominal Sets: Names and Symmetry in Computer Science. volume 57 of *Cambridge Tracts in Theoretical Computer Science*. Cambridge University Press.
- [7] Urban, C., Tasson, C., 2005. Nominal techniques in isabelle/hol, in: Nieuwenhuis, R. (Ed.), *Automated Deduction – CADE-20*, Springer Berlin Heidelberg, Berlin, Heidelberg. pp. 38–53.